

به نام خدا



هوش مصنوعی
نیم سال دوم ۰۴-۰۵
استاد: دکتر نصیری مفخم
شماره گروه: ۱۸
اعضای گروه:

فرنوش پورشبان (۴۰۲۴۰۲۳۰۰۷)
فرزانه سلیمی (۴۰۱۴۰۱۳۰۵۹)
یگانه رستگاری (۴۰۱۴۰۱۳۰۴۰)

دانشکده ریاضی و آمار

گزارش نهایی پروژه چهارم هوش مصنوعی

« حل کننده هوشمند بازی مین روب با استفاده از منطق مرتبه اول (FOL) »

پیش گفتار:

بازی مین روب (Minesweeper) یکی از مسائل کلاسیک و پرکاربرد در حوزه هوش مصنوعی، به ویژه در مبحث «عامل‌های مبتنی بر دانش (Knowledge-based Agents)» به شمار می‌رود. در این بازی، عامل در محیطی به صورت یک جدول با ابعاد مشخص قرار می‌گیرد که تعدادی از خانه‌های آن شامل مین‌های پنهان هستند. هدف عامل این است که تمامی خانه‌های امن را فاش کرده و در عین حال، تمام خانه‌های دارای مین را بدون منفجر شدن هیچ‌یک از آن‌ها به درستی پرچم گذاری کند.

چالش اصلی این مسئله در آن است که عامل در ابتدای بازی هیچ گونه آگاهی کاملی از وضعیت محیط ندارد و تنها با آشکار کردن یک خانه امن، عددی بین ۰ تا ۸ را مشاهده می‌کند که بیانگر تعداد مین‌های موجود در همسایگی همان خانه است. بنابراین، عامل باید تنها بر اساس اطلاعاتی که به تدریج در اختیارش قرار می‌گیرد، درباره وضعیت سایر خانه‌ها استدلال کرده و تصمیم‌گیری کند.

در این پروژه، به جای استفاده از الگوریتم‌های جستجوی متداول یا روش‌های صرفاً تصادفی، از منطق مرتبه اول (First-Order Logic - FOL) برای انجام فرآیند استدلال و تصمیم‌گیری استفاده شده است. در این روش، اطلاعات حاصل از خانه‌های فاش‌شده توسط محیط بازی به صورت حقایق (Facts) در اختیار موتور استنتاج منطقی (pyDatalog) قرار می‌گیرد. سپس این موتور، با تکیه بر قوانین منطقی (Rules) تعریف‌شده، به صورت کاملاً قطعی استنتاج می‌کند که کدام خانه‌ها قطعاً امن (Safe) و کدام خانه‌ها قطعاً دارای مین (Mine) هستند.

با این حال، از آنجا که در منطق کلاسیک ممکن است در برخی شرایط عامل با «بن‌بست منطقی» (Deadlock) مواجه شود؛ یعنی اطلاعات موجود برای استنتاج قطعی حرکت بعدی کافی نباشد، در این پروژه مکانیزمی برای ارزیابی هوشمند ریسک (Smart Guessing) نیز به عنوان مکمل سیستم استنتاج پیاده‌سازی شده است. این مکانیزم به عامل اجازه می‌دهد تا در شرایط عدم قطعیت، با انتخاب کم‌ریسک‌ترین گزینه ممکن، روند بازی را ادامه دهد.

لینک گیت‌هاب پروژه:

https://github.com/Farnoush2005/AI_Minesweeper-Solver-with-First-Order-Logic

تعاریف و نمادگذاری‌های پایه (Notations):

در این بخش، مجموعه‌ها، توابع و گزاره‌های پایه‌ای که در فرآیند مدل‌سازی و استنتاج منطقی مورد استفاده قرار می‌گیرند، معرفی می‌شوند.

ابتدا مجموعه خانه‌های صفحه بازی را با Grid نمایش می‌دهیم که شامل تمام خانه‌های جدول به صورت زوج مرتب (r, c) است. تابع $Adj(r, c)$ مجموعه تمام خانه‌های مجاور (r, c) را در همسایگی هشت‌گانه مشخص می‌کند و به صورت زیر تعریف می‌شود:

$$Adj(r, c) = \{(r', c') \in Grid \mid |r - r'| \leq 1 \wedge |c - c'| \leq 1 \wedge (r, c) \neq (r', c')\}$$

تابع $Val(r, c)$ عدد فاش‌شده در خانه (r, c) را برمی‌گرداند که مقدار آن در بازه ۰ تا ۸ قرار دارد.

$$Val(r, c) \in \{0, 1, \dots, 8\}$$

گزاره $Hidden(r, c)$ بیان می‌کند که خانه (r, c) هنوز فاش نشده و عامل روی آن کلیک نکرده است. گزاره $Flagged(r, c)$ نشان می‌دهد که خانه (r, c) توسط عامل به‌عنوان یک مین احتمالی پرچم‌گذاری شده است. گزاره $Safe(r, c)$ یک گزاره استنتاجی است که بیان می‌کند خانه (r, c) با قطعیت امن بوده و می‌توان آن را بدون خطر فاش کرد. در نهایت، گزاره $Mine(r, c)$ نیز یک گزاره استنتاجی است که نشان می‌دهد خانه (r, c) با قطعیت حاوی مین است.

۱. قانون امنیت (Safe Rule):

این قانون بیان می‌کند که اگر عدد درج‌شده روی یک خانه فاش‌شده دقیقاً با تعداد همسایه‌های پرچم‌گذاری‌شده آن برابر باشد، تمام خانه‌های مجاور باقی‌مانده که هنوز در وضعیت مخفی قرار دارند، با قطعیت امن هستند. در نتیجه، عامل می‌تواند این خانه‌ها را بدون خطر فاش کند.

نمایش قانون در منطق مرتبه اول (FOL)

$$\forall (r_1, c_1), (r_2, c_2) \in Grid : \left(\exists N \left[Val(r_1, c_1) = N \wedge \left(\sum_{(r', c') \in Adj(r_1, c_1)} \mathbb{I}(Flagged(r', c')) = N \right) \right] \wedge (r_2, c_2) \in Adj(r_1, c_1) \wedge Hidden(r_2, c_2) \right) \implies Safe(r_2, c_2)$$

توضیح: در این رابطه، $\mathbb{I}$ تابع نشانگر است؛ به‌طوری‌که اگر شرط داخل آن برقرار باشد، مقدار ۱ و در غیر این صورت مقدار ۰ را برمی‌گرداند.

۲. قانون خطر (Mine Rule):

این قانون بیان می‌کند که اگر عدد درج‌شده روی یک خانه فاش‌شده دقیقاً با تعداد کل خانه‌های باز نشده مجاور آن، شامل خانه‌های پنهان و خانه‌های پرچم‌گذاری‌شده، برابر باشد، تمام خانه‌های مجاور پنهانی که هنوز پرچم‌گذاری نشده‌اند، با قطعیت حاوی مین هستند.

نمایش قانون در منطق مرتبه اول (FOL)

$$\forall (r_1, c_1), (r_2, c_2) \in Grid : \left(\exists N \left[Val(r_1, c_1) = N \wedge \left(\sum_{(r', c') \in Adj(r_1, c_1)} \mathbb{I}(Hidden(r', c') \vee Flagged(r', c')) = N \right) \right] \wedge (r_2, c_2) \in Adj(r_1, c_1) \wedge Hidden(r_2, c_2) \right) \implies Mine(r_2, c_2)$$

• توضیح کد:

در این بخش، ساختار کد پیاده‌سازی شده در فایل main.py را به صورت بخش‌بخش و خط‌به‌خط بررسی می‌کنیم.

وارد کردن کتابخانه‌ها و تعریف ترم‌های منطقی (Terms Setup & Imports):

```
import pygame
import time
import random
from pyDatalog import pyDatalog
from src.minesweeper import Minesweeper

pyDatalog.create_terms('R, C, R1, C1, R2, C2, N, F, H')
pyDatalog.create_terms('Adjacent, CellValue, Safe, Mine, Hidden, FlaggedCount, HiddenCount')

AGENT_UNKNOWN = -1
AGENT_FLAGGED = -2
```

ابتدا کتابخانه‌های گرافیکی (pygame)، مدیریت زمان (time) و تولید اعداد تصادفی (random) وارد کرده و سپس هسته منطقی برنامه یعنی pyDatalog و کلاس محیط بازی (Minesweeper) اضافه شده‌اند. تابع create_terms برای معرفی متغیرها (با حروف بزرگ انگلیسی مثل R, C) تعریف کرده و گزاره‌ها/روابط منطقی (مثل Adjacent, Safe) به موتور استنتاج پایتون استفاده می‌شود. ثابت‌های AGENT_UNKNOWN و AGENT_FLAGGED برای مدیریت وضعیت داخلی برد عامل (خانه‌های پنهان یا پرچم‌گذاری شده) تعریف شده‌اند.

مدیریت پایگاه دانش پویا (Knowledge Base Management)

```
# Dictionary to keep track of facts from the previous turn so we can retract them
previous_facts = {
    'Hidden': set(),
    'CellValue': set(),
    'FlaggedCount': set(),
    'HiddenCount': set()
}
```

از آنجایی که در هر گام از بازی، وضعیت خانه‌ها تغییر می‌کند (یک خانه از حالت پنهان خارج شده و عدد آن مشخص می‌شود)، نیاز داریم که حقایق دور قبل را پاک کنیم. دیکشنری `previous_facts`، کپی حقایق ثبت شده در دور قبل را ذخیره می‌کند تا بتوانیم آن را فراخوانی و حذف کنیم.

حقایق ایستا - همسایگی (Static Facts)

```
# =====
# Logic & Rules Setup
# =====

def init_static_facts(rows, cols):
    """
    Generate static facts at the beginning of the game.
    Mainly used to define which cells are adjacent to each other.
    """
    for r1 in range(rows):
        for c1 in range(cols):
            for dr in [-1, 0, 1]:
                for dc in [-1, 0, 1]:
                    if dr == 0 and dc == 0:
                        continue
                    r2, c2 = r1 + dr, c1 + dc
                    if 0 <= r2 < rows and 0 <= c2 < cols:
                        # Using '+' to assert the Fact in pyDatalog
                        + Adjacent(r1, c1, r2, c2)
```

رابطه همسایگی (`Adjacent`) یک حقیقت ایستا است، زیرا ابعاد جدول و موقعیت خانه‌ها در طول بازی تغییر نمی‌کنند. تابع `init_static_facts` با بررسی تمام خانه‌های جدول با دو حلقه تکرار روی سطرها و ستون‌ها و بررسی ۸ جهت همسایگی، خانه‌های مجاور را شناسایی کرده و اگر خانه‌ی مقصد در محدوده‌ی جدول قرار داشته

باشد، با علامت + در پایگاه دانش pyDatalog به عنوان Fact ثبت (Assert) می کند. این کار فقط یک بار در ابتدای بازی انجام می شود تا سرعت اجرا بهینه شود.

پایه سازی قوانین استنتاج مرتبه اول (Logical Rules)

```
def init_rules():
    """
    Defining the inference rules based on the project document.
    """
    # Safe Rule:
    # If a cell's number (N) equals the number of flagged neighbors (F),
    # then all of its remaining hidden neighbors are safe to click.
    Safe(R2, C2) <= (
        CellValue(R1, C1, N) &
        FlaggedCount(R1, C1, F) &
        (N == F) &
        Adjacent(R1, C1, R2, C2) &
        Hidden(R2, C2)
    )

    # Mine Rule:
    # If a cell's number (N) equals the total number of its unrevealed
    # neighbors (H) (flags + hidden), then all those hidden neighbors are mines.
    Mine(R2, C2) <= (
        CellValue(R1, C1, N) &
        HiddenCount(R1, C1, H) &
        (N == H) &
        Adjacent(R1, C1, R2, C2) &
        Hidden(R2, C2)
    )
```

قانون امنیت (Safe Rule): در این قانون، عامل ابتدا یک خانه ی باز شده (R_1, C_1) را بررسی می کند که مقدار آن برابر با N است. سپس تعداد خانه هایی که قبلاً به عنوان مین علامت گذاری شده اند (F) را در اطراف آن خانه محاسبه می کند. اگر تعداد پرچم های قرار گرفته دقیقاً با عدد روی خانه برابر باشد ($N == F$) به این معناست که تمام مین های مرتبط با آن خانه از قبل شناسایی شده اند و مین ناشناخته ی دیگری در همسایگی آن باقی نمانده است. بنابراین می توان از هر خانه ی مجاور (R_2, C_2) که هنوز در وضعیت Hidden قرار دارد، با به عنوان یک خانه ی Safe محسوب کرد و عامل می تواند بدون خطر آن را باز کند. این قانون در واقع از این حقیقت استفاده می کند که وقتی همه ی مین های مورد انتظار یک خانه مشخص شده باشند، سایر همسایه های باز نشده ی آن الزاماً امن هستند.

قانون خطر (Mine Rule): در این قانون نیز عامل از یک خانه‌ی بازشده (R_1, C_1) با مقدار N شروع می‌کند، اما این بار تعداد کل خانه‌های بازنشده‌ی اطراف آن را (شامل خانه‌های پنهان و همچنین خانه‌های پرچم‌گذاری‌شده)، با مقدار H در نظر می‌گیرد. اگر مقدار عدد روی خانه دقیقاً با تعداد این خانه‌های بازنشده برابر باشد ($N == H$)، نتیجه می‌گیریم که تمام این خانه‌ها باید دارای مین باشند؛ چرا که هنوز به تعداد مورد نیاز مین شناسایی نشده است و تنها مکان‌های ممکن برای وجود آن‌ها، در همین خانه‌های بازنشده است. در نتیجه، هر خانه‌ی مجاور (R_2, C_2) که همچنان در وضعیت Hidden قرار دارد، به عنوان یک Mine محسوب می‌شود و عامل آن را برای جلوگیری از انتخاب اشتباه، به عنوان مین علامت‌گذاری می‌کند. این قانون به عامل اجازه می‌دهد بدون حدس زدن و صرفاً بر اساس روابط منطقی میان اعداد و خانه‌های بازنشده، محل مین‌ها را با قطعیت شناسایی کند.

به‌روزرسانی پویا و استخراج حقایق (Dynamic Fact Update)

این بخش وظیفه دارد دنیای واقعی بازی (وضعیت جدول) Pygame را ترجمه کرده و به زبان منطق قابل فهم برای موتور pyDatalog درآورد.

```
def update_knowledge_base(agent_board, rows, cols):  
    """  
    Translates the current board state into dynamic facts.  
    """  
    global previous_facts
```

به‌روزرسانی پایگاه دانش (update_knowledge_base): این تابع وضعیت فعلی جدول عامل (`agent_board`) را دریافت کرده و آن را به مجموعه‌ای از حقایق پویا برای پایگاه دانش تبدیل می‌کند. همچنین با استفاده از متغیر سراسری `previous_facts`، حقایقی که در مرحله‌ی قبل ثبت شده‌اند را نگه می‌دارد تا ثبت دوباره‌ی اطلاعات تکراری جلوگیری کند.

```
# 1. Retract old facts from the previous iteration (using '-')
for r, c in previous_facts['Hidden']:
    - Hidden(r, c)
for r, c, val in previous_facts['CellValue']:
    - CellValue(r, c, val)
for r, c, f in previous_facts['FlaggedCount']:
    - FlaggedCount(r, c, f)
for r, c, h in previous_facts['HiddenCount']:
    - HiddenCount(r, c, h)
```

در این بخش، قبل از اضافه کردن اطلاعات جدید، حقایق مربوط به وضعیت قبلی جدول از پایگاه دانش حذف می‌شوند. در pyDatalog برای حذف یک Fact از عملگر منفی پیشوندی (-) استفاده می‌شود که مشابه عملیات retract در زبان Prolog عمل می‌کند. با اجرای این حلقه‌ها، اطلاعات قبلی مربوط به خانه‌های پنهان (Hidden)، مقادیر خانه‌ها (CellValue)، تعداد مین‌های علامت‌گذاری‌شده (FlaggedCount) و تعداد خانه‌های باز نشده (HiddenCount) از حافظه‌ی موتور منطقی پاک می‌شوند. این کار باعث می‌شود پایگاه دانش با وضعیت جدید جدول هماهنگ بماند و از باقی ماندن اطلاعات قدیمی یا ایجاد تناقض در فرآیند استنتاج جلوگیری شود.

```
# Reset the tracking dictionary
previous_facts = {
    'Hidden': set(),
    'CellValue': set(),
    'FlaggedCount': set(),
    'HiddenCount': set()
}
```

در این بخش دیکشنری بافر را خالی می‌کنیم تا در مرحله بعد، حقایق جدیدی که ثبت می‌کنیم را درون آن بنویسیم.

```
# 2. Assert new facts based on the updated board
for (r, c), val in agent_board.items():
    if val == AGENT_UNKNOWN:
        + Hidden(r, c)
        previous_facts['Hidden'].add((r, c))
    elif val >= 0:
        + CellValue(r, c, val)
        previous_facts['CellValue'].add((r, c, val))
```

سپس تمام خانه‌های موجود در برد عامل بررسی می‌شوند تا اطلاعات جدید آن‌ها در پایگاه دانش ثبت شود. اگر مقدار یک خانه برابر AGENT_UNKNOWN باشد، به این معناست که خانه هنوز باز نشده است؛ بنابراین حقیقت

Hidden(r, c) با استفاده از عملگر + در پایگاه دانش ثبت شده و مختصات آن در previous_facts ذخیره می‌شود. در مقابل، اگر مقدار خانه بزرگ‌تر یا مساوی صفر باشد (خانه قبلاً باز شده) حقیقت CellValue(r, c, val) آن را با مقدار عددی خانه در پایگاه دانش ثبت شده و اطلاعات آن نیز در previous_facts نگه‌داری شده تا در به‌روزرسانی‌های بعدی قابل مدیریت باشد.

```
# Count flagged and completely hidden neighbors
flagged = 0
hidden = 0
for dr in [-1, 0, 1]:
    for dc in [-1, 0, 1]:
        if dr == 0 and dc == 0: continue
        nr, nc = r + dr, c + dc
        if (nr, nc) in agent_board:
            n_val = agent_board[(nr, nc)]
            if n_val == AGENT_FLAGGED:
                flagged += 1
            elif n_val == AGENT_UNKNOWN:
                hidden += 1
```

محاسبه تعداد همسایه‌های پرچم‌گذاری شده و پنهان:

قوانین استنتاج ما (Safe Rule و Mine Rule) نیاز دارند تعداد همسایه‌های پنهان و پرچم‌گذاری شده اطراف هر عدد را بدانند. این اطلاعات باید به صورت عددی محاسبه شده و به عنوان یک حقیقت منطقی ثبت شوند. برای هر خانه عدددار فاش شده، یک همسایگی ۸ گانه (با تغییر سطر و ستون از -۱ تا +۱) ایجاد می‌شود. سپس بررسی می‌کنیم که چه تعداد از این همسایه‌ها پرچم خورده‌اند (flagged) و چه تعداد هنوز پنهان هستند (hidden).

در فرآیند پیمایش، ابتدا وجود همسایه در محدوده برد عامل (agent_board) تایید می‌گردد و سپس وضعیت مقدار آن استخراج می‌شود. اگر مقدار خانه‌ی همسایه برابر با AGENT_FLAGGED باشد، به این معناست که آن خانه به عنوان مین شناسایی شده است؛ پس شمارنده flagged یک واحد افزایش می‌یابد. به همین ترتیب اگر مقدار خانه برابر با AGENT_UNKNOWN باشد، به این معناست که آن خانه هنوز باز نشده و وضعیت آن ناشناخته است؛ بنابراین، شمارنده hidden به‌روزرسانی می‌شود. مقادیر نهایی حاصل از این شمارش، به عنوان حقایق عددی در اختیار موتور استنتاج قرار می‌گیرند تا بر اساس آن‌ها، تحلیل‌های بعدی بر روی خانه‌های عدددار انجام شود.

```

+ FlaggedCount(r, c, flagged)
previous_facts['FlaggedCount'].add((r, c, flagged))

# H = total unrevealed neighbors (hidden + flagged)
total_unrevealed = hidden + flagged
+ HiddenCount(r, c, total_unrevealed)
previous_facts['HiddenCount'].add((r, c, total_unrevealed))

```

ثبت حقایق در پایگاه دانش: پس از اتمام فرآیند شمارش، مقادیر به دست آمده باید به عنوان حقایق منطقی در پایگاه دانش ثبت شوند تا موتور استنتاج بتواند از آن‌ها برای تصمیم‌گیری استفاده کند. تعداد همسایه‌های پرچم‌گذاری شده با استفاده از گزاره `FlaggedCount(r, c, flagged)` ثبت می‌گردد. علاوه بر این، برای اعمال قوانین استنتاجی، نیاز است که تعداد کل خانه‌های «بازنشده» (`Unrevealed`) اطراف هر خانه مشخص باشد. نماد `H` (طبق داک پروژ) استفاده شده که شامل مجموع خانه‌های پنهان (`hidden`) و خانه‌های پرچم‌گذاری شده (`flagged`) می‌باشد. این مقدار با متغیر `total_unrevealed` محاسبه شده و با استفاده از گزاره `HiddenCount(r, c, total_unrevealed)` به پایگاه دانش اضافه می‌شود. ثبت این حقیقت، برای اجرای `Mine Rule` مهم است چرا که از این مقدار برای شناسایی دقیق موقعیت مین‌ها استفاده می‌کند.

پرس‌وجو از موتور منطقی (Querying)

```

def query_solver():
    """
    Query the pyDataLog engine to find safe moves and mines.
    """
    safe_moves = set()
    mine_moves = set()

    # Query for safe cells
    ans_safe = Safe(R, C)
    if ans_safe:
        for r, c in ans_safe.data: # .data is safer to iterate over
            safe_moves.add((r, c))

    # Query for definitely mine cells
    ans_mine = Mine(R, C)
    if ans_mine:
        for r, c in ans_mine.data:
            mine_moves.add((r, c))

    return list(safe_moves), list(mine_moves)

```

استخراج نتایج از موتور استنتاج: پس از ثبت تمامی حقایق استخراج شده در پایگاه دانش، عامل با پرسیدن پرسش‌های استنتاجی به دنبال پاسخ برای این سؤال است که: «کدام جفت از مختصات (R, C) ، (R, C) ، (R, C) ، (R, C) Safe(R, C) یا Mine(R, C) را ارضا می‌کنند؟».

برای این منظور، از ساختار داده‌ای **data** استفاده می‌شود؛ این رویکرد به ما این امکان را می‌دهد که به طور مستقیم، سریع و بدون خطا، به مجموعه‌ی پاسخ‌های استخراج شده توسط موتور منطقی دسترسی داشته باشیم. نتایج نهایی به صورت یک لیست از مختصات به عامل بازگردانده می‌شوند تا عامل بتواند بر اساس آن‌ها، حرکت بعدی خود را به درستی انتخاب کند.

مدیریت بن بست منطقی با حدس هوشمند (Smart Guessing)

زمانی که عامل هوشمند به بن بست منطقی (Logical Deadlock) می‌رسد، باید خانه‌ای را انتخاب کند که کمترین احتمال وجود مین را داشته باشد. برای این کار، ما خانه‌ها را به دو دسته تقسیم می‌کنیم:

۱. **خانه‌های مرزی (Fringe Cells):** خانه‌های پنهانی که حداقل با یک خانه فاش شده‌ی عدددار همسایه هستند. این خانه‌ها دارای اطلاعات احتمالاتی هستند.

۲. **خانه‌های ایزوله (Isolated Cells):** خانه‌های پنهانی که در اطراف آن‌ها هیچ عدد فاش شده‌ای وجود ندارد و هیچ اطلاعات محلی از آن‌ها در دست نیست.

برای محاسبه ریسک یک خانه مرزی مانند (h_r, h_c) که در مجاورت خانه عدددار (n_r, n_c) با عدد N قرار دارد، فرمول زیر را به کار می‌بریم:

$$Risk = \frac{N - F}{U}$$

که در آن:

- N عدد نوشته شده روی خانه فاش شده است (تعداد کل مین های همسایه).
 - F تعداد پرچم هایی است که در همسایگی آن خانه گذاشته شده است.
 - U تعداد همسایه های پنهان و بدون پرچم آن خانه است.
 - تفاضل $N - F$ نشان دهنده تعداد مین های باقی مانده و کشف نشده در اطراف آن عدد است.
- اگر یک خانه پنهان با چند خانه عدددار همسایه باشد، ما بدترین ریسک (Maximum Risk) را برای آن خانه در نظر می گیریم (رویکرد محافظه کارانه یا Pessimistic) و در نهایت خانه ای را انتخاب می کنیم که این ریسک بیشینه اش، از همه کمتر باشد (رویکرد Minimax).

```
# =====  
# Deadlock Handling (Smart Guessing)  
# =====  
  
def get_safest_guess(agent_board, rows, cols):  
    """  
    When FOL gets stuck, we need to guess.  
    This calculates a simple risk factor for fringe cells to pick the safest bet.  
    """  
    best_guess = None  
    min_risk = float('inf')
```

ابتدا متغیر `best_guess` را برای ذخیره بهترین مختصات کاندید حدس و `min_risk` را با مقدار بی نهایت مثبت (`inf`) مقداردهی اولیه می کنیم تا هر ریسکی که محاسبه می شود از آن کمتر باشد و جایگزینش شود.

```
hidden_cells = [(r, c) for (r, c), v in agent_board.items() if v == AGENT_UNKNOWN]  
if not hidden_cells:  
    return None
```

با استفاده از یک List Comprehension، تمام خانه هایی که در برد عامل وضعیت آن ها هنوز نامشخص

(AGENT_UNKNOWN یا همان ۱-) است را پیدا کرده و در لیست hidden_cells ذخیره می کنیم. اگر هیچ

خانه پنهانی باقی نمانده باشد، بازی تمام است و تابع مقدار None بازمی گرداند.

```
for hr, hc in hidden_cells:
    max_adj_risk = 0.0
    is_fringe = False
```

برای تک تک خانه های پنهان جدول یک حلقه می سازیم. متغیر max_adj_risk را برابر صفر قرار می دهیم تا بیشترین ریسک تحمیل شده به این خانه از سمت همسایه هایش را محاسبه کنیم. پرچم is_fringe نیز بررسی می کند که آیا این خانه پنهان، یک خانه مرزی (در مجاورت اعداد) است یا خیر.

```
for dr in [-1, 0, 1]:
    for dc in [-1, 0, 1]:
        if dr == 0 and dc == 0: continue
        nr, nc = hr + dr, hc + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            n_val = agent_board.get((nr, nc))
            if n_val is not None and n_val >= 0:
                is_fringe = True
```

این بخش تمام ۸ همسایه اطراف خانه پنهان فعلی را بررسی می کند تا ببیند آیا همسایه فاش شده ای (عدددار) دارد یا خیر. اگر خانه همسایه معتبر باشد، مقدار آن در n_val ذخیره می شود. اگر مقدار همسایه بزرگتر یا مساوی صفر باشد، یعنی این یک خانه فاش شده با یک عدد معین است. پس خانه پنهان ما رسماً یک خانه مرزی است و is_fringe برابر True می شود.

```

flagged = 0
unknown = 0
for ddr in [-1, 0, 1]:
    for ddc in [-1, 0, 1]:
        if ddr == 0 and ddc == 0: continue
        nnr, nnc = nr + ddr, nc + ddc
        if 0 <= nnr < rows and 0 <= nnc < cols:
            v = agent_board.get((nnr, nnc))
            if v == AGENT_FLAGGED:
                flagged += 1
            elif v == AGENT_UNKNOWN:
                unknown += 1

```

حال که یک همسایه عدددار پیدا کردیم، باید ببینیم وضعیت اطراف خود آن همسایه عدددار چگونه است. برای این کار، یک حلقه داخلی تر روی همسایه‌های آن خانه عدددار می‌زنیم تا تعداد پرچم‌ها (flagged) و تعداد خانه‌های پنهان باقی‌مانده (unknown) را بشماریم. این شمارش به ما اجازه می‌دهد مخرج و صورت کسر احتمال را به دست آوریم.

```

if unknown > 0:
    # Risk = remaining mines around cell / remaining unknown cells
    risk = (n_val - flagged) / unknown
    if risk > max_adj_risk:
        max_adj_risk = risk

```

در این خط، فرمول ریاضی ریسک پیاده‌سازی شده است با محاسبه‌ی تفاضل میان مقدار عدد خانه و تعداد پرچم‌های موجود، تقسیم بر تعداد کل خانه‌های باز نشده در همسایگی آن. از آنجایی که یک خانه پنهان ممکن است بین چند عدد محاصره شده باشد، ما در هر مرحله بیشترین ریسک محاسبه شده را در max_adj_risk ذخیره می‌کنیم تا بدترین سناریو را مبنا قرار دهیم.

```

# Only evaluate fringe cells (cells touching the numbers)
if is_fringe:
    if max_adj_risk < min_risk:
        min_risk = max_adj_risk
        best_guess = (hr, hc)

```

پس از پایان بررسی همسایه‌ها برای خانه پنهان فعلی، اگر این خانه مرزی بود، ریسک آن را با کمترین ریسک ثبت شده کل جدول (min_risk) مقایسه می‌کنیم. اگر ریسک این خانه از رکوردهای قبلی کمتر بود، این خانه به

عنوان امن ترین حدس تا این لحظه در `best_guess` ذخیره شده و ریسک آن به عنوان حد نصاب جدید ثبت می شود.

```
if best_guess:
    return best_guess

# If no fringe cells are found, just pick a random one
return random.choice(hidden_cells)
```

در نهایت، اگر پس از بررسی تمام خانه ها موفق شدیم یک خانه مرزی با کمترین ریسک ممکن پیدا کنیم، مختصات آن را بازمی گردانیم تا انتخاب شود. اما اگر هیچ خانه مرزی پیدا نشد (به عنوان مثال در ابتدای بازی که کل جدول پوشیده است و هیچ عددی فاش نشده)، تابع یک خانه را به طور کاملاً تصادفی با `random.choice` انتخاب می کند تا بازی شروع شود.

حلقه اصلی بازی و کنترل کننده

```
# =====
# Main Game Loop
# =====

def prolog_solver(game):
    # Clear pyDatalog completely so it's fresh for each test case
    pyDatalog.clear()
```

قبل از هر چیز، با متد `pyDatalog.clear()` تمام متغیرها، قوانین و حقایق قدیمی موجود در حافظه مفسر پاکسازی می شوند. این کار به این دلیل است که اگر چند سناریوی تست را پشت سر هم اجرا کنیم، داده های اجرای قبلی روی اجرای جدید اثر نگذارند.

```

global previous_facts
previous_facts = {
    'Hidden': set(), 'CellValue': set(), 'FlaggedCount': set(), 'HiddenCount': set()
}

# Initial setup
init_static_facts(game.rows, game.cols)
init_rules()

```

بافر ذخیره حقایق را reset می کنیم. سپس حقایق ایستا (رابطه مجاورت خانه‌ها) را با متد `init_static_facts` می سازیم و قوانین منطق مرتبه اول (Safe Rule و Mine Rule) را با متد `init_rules` کامپایل می کنیم.

```

agent_board = {}
for r in range(game.rows):
    for c in range(game.cols):
        agent_board[(r, c)] = AGENT_UNKNOWN

```

برد داخلی عامل (پایگاه دانش ذهنی ربات ما از بازی) مقداردهی اولیه می شود. در ابتدا ربات هیچ خانه‌ای را باز نکرده است، پس وضعیت تمام مختصات‌ها برابر `AGENT_UNKNOWN` (یا همان پنهان) تنظیم می شود.

```

start_r, start_c = game.get_start_pos()
print(f"Starting at guaranteed safe position: {start_r}, {start_c}")

# First move is always safe based on environment constraints
start_val = game.reveal(start_r, start_c)
agent_board[(start_r, start_c)] = start_val if start_val is not None else 0

```

بر اساس قوانین بازی، اولین خانه باز شده هرگز نباید دارای مین باشد. متد `get_start_pos` از محیط بازی، یک خانه ۱۰۰٪ امن (معمولاً با مقدار ۰) را به عنوان نقطه آغازین به عامل تحویل می دهد. عامل این خانه را کلیک کرده (`reveal`) و مقدار عدد آن را در برد ذهنی خود ثبت می کند تا استنتاج از همین نقطه کلید بخورد.


```

running = True
while running and not game.game_over:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

```

این حلقه تا زمانی که بازی تمام نشود (not game.game_over) یا کاربر پنجره بازی را نبندد ادامه دارد. هندلر رویدادهای Pygame نیز برای جلوگیری از فریز شدن پنجره و مدیریت بسته شدن آن قرار داده شده است.

```

# 1. Update logic
update_knowledge_base(agent_board, game.rows, game.cols)

# 2. Inference
safe_moves, mine_moves = query_solver()
move_made = False

```

در قسمت ۱، ابتدا با استفاده از وضعیت جدید برد، پایگاه دانش منطقی را به‌روزرسانی می‌کنیم. در قسمت ۲، تابع query_solver فعال شده و از موتور منطقی دو سوال اساسی می‌پرسد: "کدام خانه‌ها بر اساس فرمول‌های منطق مرتبه اول امن هستند؟" و "کدام خانه‌ها قطعاً مین هستند؟". خروجی‌ها در قالب دو لیست دریافت می‌شوند.

```

# 3. Apply moves (Flag mines first, then reveal safe cells)
for r, c in mine_moves:
    if agent_board[(r, c)] == AGENT_UNKNOWN:
        game.flag(r, c)
        agent_board[(r, c)] = AGENT_FLAGGED
        move_made = True

for r, c in safe_moves:
    if agent_board[(r, c)] == AGENT_UNKNOWN:
        val = game.reveal(r, c)
        agent_board[(r, c)] = val if val is not None else 0
        move_made = True

```

پس از تعیین خانه‌های امن و مین‌ها توسط موتور استنتاج، عامل حرکات خود را با اولویت‌بندی استراتژیک اعمال می‌کند. اولویت اول همیشه با پرچم‌گذاری مین‌ها (Flagging) است؛ چرا که با مشخص شدن موقعیت دقیق مین‌ها، فضای جستجو محدودتر شده و فرآیند استنتاج سریع‌تر می‌شود. در این مرحله، مختصات mine_moves بررسی

و وضعیت خانه‌های پنهان در `agent_board` به `AGENT_FLAGGED` تغییر می‌یابد. سپس، عامل خانه‌های موجود در `safe_moves` را با استفاده از تابع `game.reveal` باز کرده و مقدار جدید آن‌ها را بلافاصله در `agent_board` ثبت می‌کند تا در محاسبات بعدی در دسترس باشد. در نهایت، اگر تغییری در وضعیت محیط (فاش شدن یا پرچم‌گذاری خانه‌ها) رخ دهد، متغیر `move_made` برابر با `True` قرار می‌گیرد که نشان‌دهنده‌ی انجام یک حرکت منطقی در این گام است.

```
# 4. Handle Deadlocks (if no logical moves are found)
if not move_made and not game.game_over:
    print("Logical deadlock! Attempting a smart guess...")
    guess_move = get_safest_guess(agent_board, game.rows, game.cols)
    if guess_move:
        r, c = guess_move
        val = game.reveal(r, c)
        agent_board[(r, c)] = val if val is not None else 0
```

مدیریت بن‌بست: گاهی منطق محض به بن‌بست منطقی می‌رسد یعنی اطلاعات برای استنتاج قطعی کافی نیستند و `move_made` همچنان `False` باقی می‌ماند. در این شرایط، عامل برای عبور از بن‌بست متوقف نمی‌شود بلکه از مکانیزم حدس هوشمند استفاده می‌کند و تابع ارزیابی ریسک (`get_safest_guess`) فراخوانی می‌شود تا خانه‌ای با کمترین احتمال وجود مین باز شود. این رویکرد ریسک شکست را به حداقل می‌رساند.

```
game.render()
time.sleep(0.05) # Delay for better visualization
```

در انتهای هر چرخه، برد بازی با تغییرات جدید روی صفحه مانیتور `render` می‌شود. یک تاخیر بسیار کوتاه ۵۰ میلی‌ثانیه‌ای هم قرار داده شده تا چشمان کاربر بتواند استدلال مرحله‌به‌مرحله و پویای عامل منطقی را مشاهده کند و بازی در یک چشم به هم زدن تمام نشود.

```
# Keep the window open when the game ends
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    game.render()
```

پس از اتمام بازی (برد یا باخت)، برای اینکه پنجره سریعاً بسته نشود و کاربر بتواند نتیجه نهایی و آمار بازی (تعداد مین‌های درست پرچم‌گذاری شده و...) را روی صفحه ببیند، یک حلقه انتظار گرافیکی ساده قرار داده شده است که تا زمان کلیک روی دکمه خروج پنجره را باز نگه می‌دارد.

• خروجی‌ها:

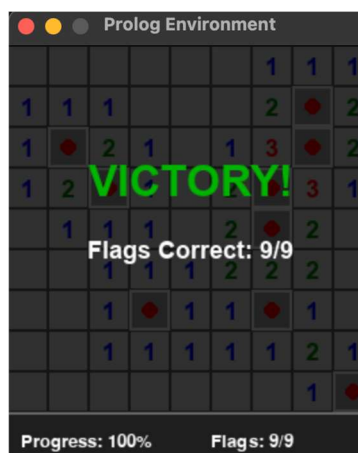
برای تست پروژه در هر کدام از سناریوها و سطح‌های مشخص شده در داکيومنت، متغیرهای بخش زیر را تغییر می‌دهیم:

```
if __name__ == "__main__":
```

۱. سناریوی ساده (تست منطق - Simple)

این سناریو برای این است که مطمئن شویم قوانین منطقی پایه به‌درستی کار می‌کنند. انتظار می‌رود که عامل هوشمند بازی را به‌طور کامل و بدون باخت حل کند.

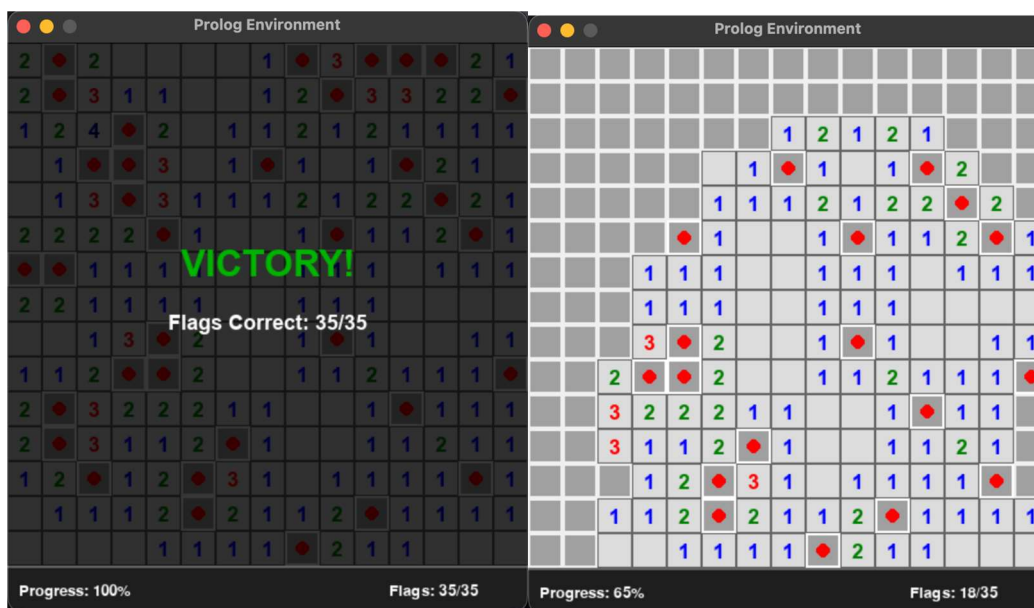
```
if __name__ == "__main__":
    ms = Minesweeper(rows=9, cols=9, mines=9, seed=99, auto_flood_fill=False)
    prolog_solver(ms)
```



۲. سناریوی استاندارد (Standard)

این تست اصلی برای ارزیابی قدرت حل مسئله عامل هوشمند در ابعاد متوسط است. انتظار می‌رود که عامل هوشمند بازی را به‌طور کامل و بدون باخت حل کند.

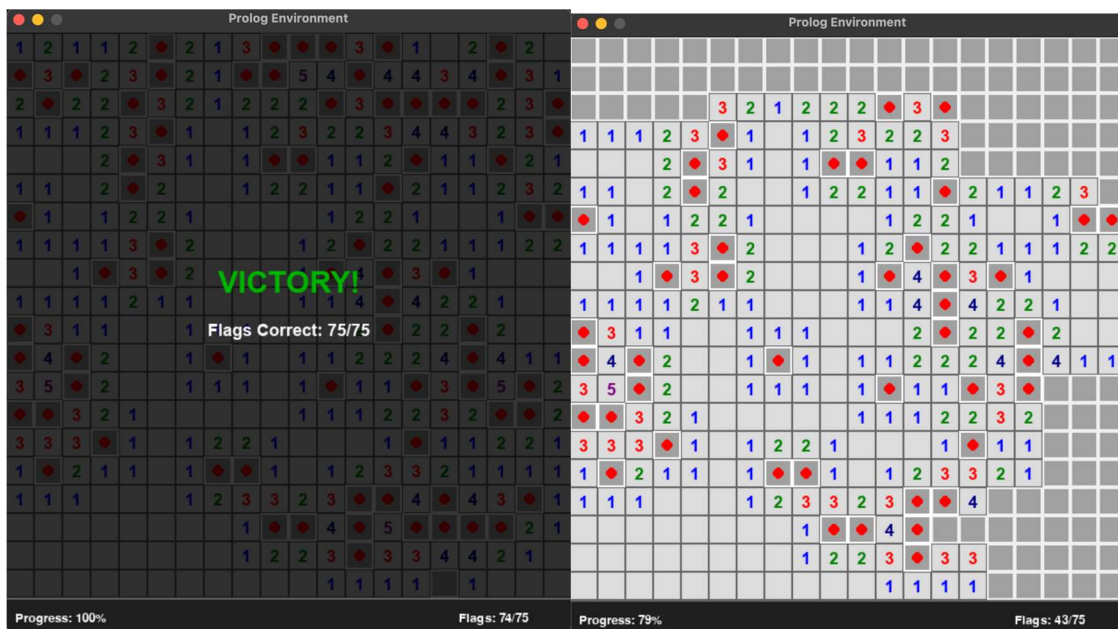
```
if __name__ == "__main__":  
    ms = Minesweeper(rows=15, cols=15, mines=35, seed=42, auto_flood_fill=False)  
    prolog_solver(ms)
```



۳. سناریوی چالشی (Challenging)

در این سناریو به دلیل چگالی بالای مین‌ها، موقعیت‌هایی پیش می‌آید که هیچ استنتاج منطقی قطعی وجود ندارد (شانس ۵۰-۵۰). **انتظار می‌رود که حداقل ۸۰٪ پیشرفت** در بازی داشته باشیم. با وجود شانس ۵۰-۵۰ باختن در این بخش ایرادی ندارد، اما می‌خواهیم عامل حداکثر تلاشش را کند.

```
if __name__ == "__main__":  
    ms = Minesweeper(rows=20, cols=20, mines=75, seed=123, auto_flood_fill=False)  
    prolog_solver(ms)
```

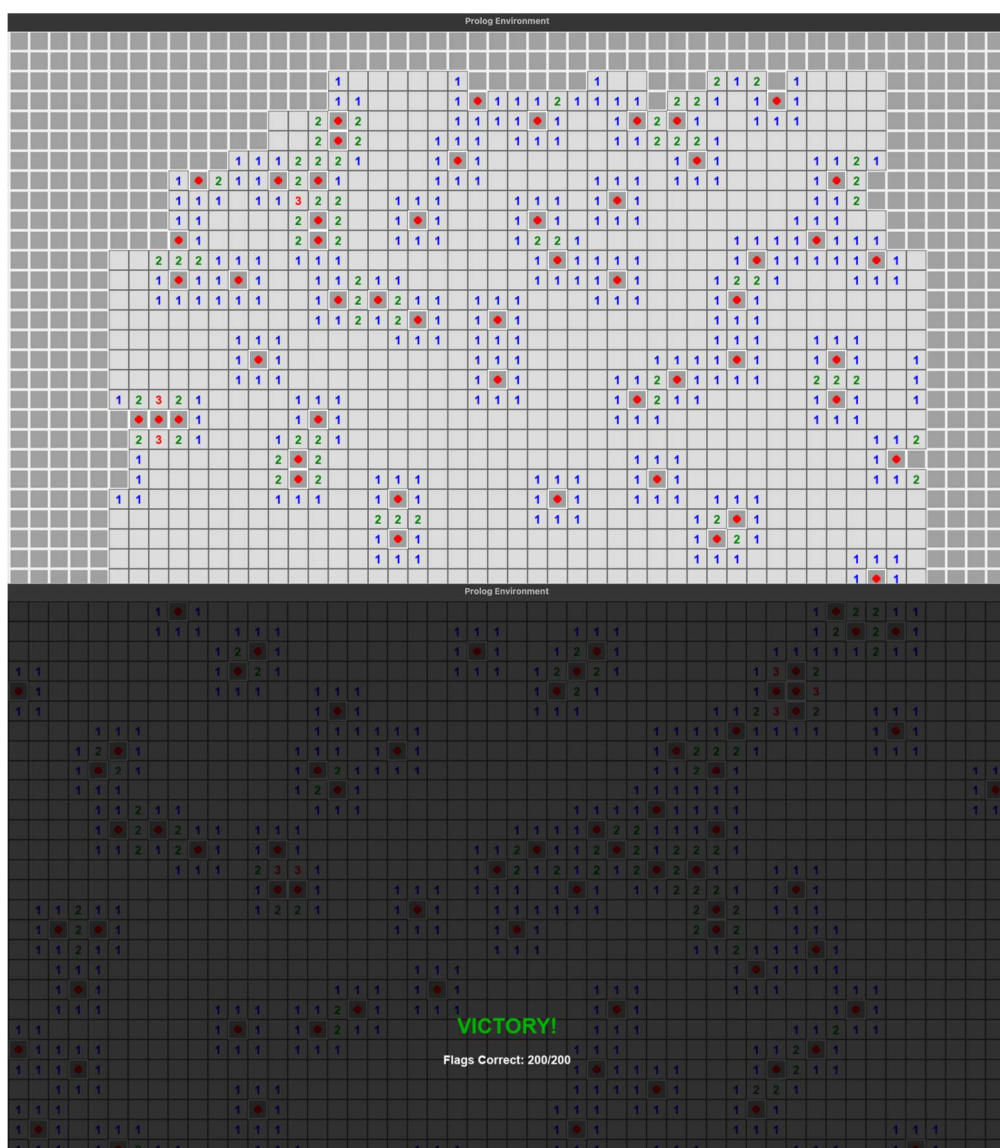


۴. سناریوی بزرگ (تست سرعت - Large)

این سناریو برای تست کارایی و بهینه بودن کدها است تا مطمئن شوند برنامه کرش نمی کند یا به خاطر پر شدن حافظه کند نمی شود. **انتظار می رود** برنامه بدون کرش کردن یا افت سرعت شدید به اجرای خود ادامه دهد.

مقدار **seed=-1** در کد زیر باعث می شود نقشه در هر بار اجرا به صورت کاملاً تصادفی تولید شود.

```
if __name__ == "__main__":  
    ms = Minesweeper(rows=45, cols=80, mines=200, seed=-1, auto_flood_fill=False)  
    prolog_solver(ms)
```



- جمع‌بندی (Conclusion):

پروژه پیاده‌سازی شده به خوبی نشان می‌دهد که چگونه منطق مرتبه اول (FOL) می‌تواند در بازی‌های مبتنی بر استدلال شرایط فاش شده، نقش مهمی ایفا کند. استفاده از تفکیک حقایق به دو دسته‌ی ایستا (Static) و پویا (Dynamic) و به کارگیری مکانیزم کارآمد **Retract** و **Assert**، باعث شده است که عامل هوشمند بدون افت فریم گرافیکی و با سرعت بسیار بالا پردازش‌های منطقی را انجام دهد.

نتایج آزمایش‌ها در سناریوی استاندارد ارزیابی نیز نشان داد که عامل، با ترکیب استنتاج قطعی مبتنی بر منطق کلاسیک و مدیریت احتمالاتی ریسک در شرایط بن‌بست، می‌تواند جدول بازی را به صورت کامل و بدون خطا حل کند. این معماری نمونه‌ای از یک الگوی استاندارد و مهندسی شده برای طراحی سیستم‌های خبره هوشمند در محیط‌های دارای اطلاعات ناقص (Incomplete Information) محسوب می‌شود.